“Computer Vision Lip Reading System Using 3D Convolutional Neural Networks”

A PROJECT REPORT

Submitted by

PATEL PRIYANSHI [23BEIT30143]

PATEL SHREYA [23BEIT30150]

In fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

INFORMATION TECHNOLOGY

LDRP Institute of Technology and Research, Gandhinagar

Kadi Sarva Vishwavidyalaya

April-2025-26

LDRP INSTITUTE OF TECHNOLOGY AND RESEARCH

GANDHINAGAR

IT Department

CERTIFICATE

This is to certify that the Project Work entitled **“Computer Vision Lip Reading System Using 3D Convolutional Neural Networks”** has been carried out by **PATEL PRIYANSHI [23BEIT30143]** under my guidance in fulfilment of the degree of Bachelor of Engineering in Information Technology Semester-6 of Kadi Sarva Vishwavidyalaya during the academic year 2025-2026.

Prof.Palak Parmar	**Dr.Mehul Barot**
Internal Guide	**Head of the Department**
LDRP ITR	**LDRP ITR**

LDRP INSTITUTE OF TECHNOLOGY AND RESEARCH

GANDHINAGAR

IT Department

CERTIFICATE

This is to certify that the Project Work entitled **"Computer Vision Lip Reading System Using 3D Convolutional Neural Networks"** has been carried out by **PATEL**

SHREYA [23BEIT301450]under my guidance in fulfilment of the degree of Bachelor of Engineering in Information Technology Semester-6 of Kadi Sarva Vishwavidyalaya during the academic year 2025-2026.

Prof.Palak Parmar

Internal Guide

LDRP ITR

Dr.Mehul Barot

Head of the Department

LDRP ITR

Presentation-I for Project-I

1. Name&Signature of Internal Guide	
2. Comments from Panel Members	

3. Name&Signature of Panel **Members**	

<u>Presentation-II for Project-I</u>

1. Name&Signature of Internal **Guide**	

2. Comments from Panel Members	
3. Name&Signature of Panel Members	

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have supported us in the successful completion of the project "Computer Vision Lip Reading System Using 3D Convolutional Neural Networks".

First and foremost, we extend our heartfelt thanks to our project guide PROF. Palak Parmar & Head of the department DR. Mehul Barot whose valuable guidance, encouragement, and constructive suggestions have been a constant source of inspiration throughout this work.

We are also thankful to the IT Department, LDRP-ITR for providing us with the facilities and resources required to carry out this project.

Our deep appreciation goes to our classmates, friends, and family for their continuous motivation and support during this journey.

Finally, we acknowledge with gratitude all the researchers and developers whose contributions in the fields of Computer Vision, Deep Learning, Speech Recognition, and Facial Landmark Detection have provided us with insights and references to complete this project successfully.

This project has been a great learning experience, and we are truly grateful for the opportunity to explore, design, and implement this Computer Vision Lip Reading System.

PATEL PRIYANSHI [23BEIT30143]

PATEL SHREYA [23BEIT30150]

ABSTRACT

This project presents a Computer Vision-based Lip Reading System that leverages 3D Convolutional Neural Networks (3D CNNs) to recognize spoken words by analyzing lip movements captured through video. Unlike traditional speech recognition systems that rely on audio signals, this system operates entirely on visual input, making it applicable in noisy environments, for hearing-impaired individuals, and in scenarios where audio-based recognition is impractical.

The system is trained on a custom-built dataset consisting of approximately 700 video clips totaling around 3 GB of data, covering 13 predefined English words: "here", "is", "a", "demo", "can", "you", "read", "my", "lips", "cat", "dog", "hello", and "bye". The dataset was manually collected using a webcam-based data collection pipeline that detects when a person is speaking by measuring the distance between upper and lower lip landmarks using dlib's facial landmark detector. Each captured word is represented as a sequence of 22 preprocessed lip-region frames, each sized 80x112 pixels in RGB color space.

The core model architecture is a 3D Convolutional Neural Network built using TensorFlow and Keras, featuring three Conv3D layers (with 8, 32, and 256 filters respectively), MaxPooling3D layers, L2 regularization, and multiple fully connected Dense layers with Dropout for regularization. The model was trained using the Adam optimizer with categorical cross-entropy loss over 20 epochs, achieving a training accuracy of 95.7% and a validation accuracy of 98.5%, demonstrating strong classification performance across all 13 word classes.

TABLE OF CONTENTS

NO	CHAPTER NAME	PAGE NO
1	INTRODUCTION	1
	1.1 Introduction	2
	1.2 Aims and Objective of the Work	4
	1.2.1 Aims	4
	1.2.2 Objectives	4
	1.3 Brief Literature Review	6
	1.3.1 Visual Speech Recognition	6
	1.3.2 Deep Learning for Lip Reading	6
	1.3.3 Facial Landmark Detection	6
	1.4 Problem Definition	7
	1.5 Plan of the Work	8
2	TECHNOLOGY AND LITERATURE REVIEW	10
	2.1 Technologies	11
	2.1.1 Programming Language & Runtime	11

2.1.2 Deep Learning Framework 11

2.1.3 Computer Vision Libraries 11

2.1.4 Data Processing & Utilities 12

2.2 Literature Review 13

2.2.1 Contextual Background 13

2.2.2 Existing Solutions & Market Analysis 13

2.3 Problem Statement&Gaps 14

2.4 The Role of Deep Learning Intervention (Project's Contribution) 14

3 SYSTEM REQUIREMENTS STUDY 17

3.1 User Characteristics 18

3.1.1 Hearing-Impaired Individuals (Primary Users) 18

3.1.2 Researchers & Developers (Secondary Users) 18

3.2 Hardware and Software Requirements 20

3.2.1 Hardware Requirements 20

3.2.2 Software Requirements 20

3.3 Assumptions and Dependencies 21

3.3.1 Assumptions 21

3.3.2 Dependencies 21

4	SYSTEM DIAGRAMS	23
	4.1 Entity-Relationship Diagram	24
	4.2 Class Diagram	25
	4.3 Use Case Diagram	25
	4.4 Sequence Diagram	
	4.5 Activity Diagram	28
	4.6 Data Flow Diagram	29
	4.6.1 Context-Level-0 DFD	29
	4.6.2 Context-Level-1 DFD	29
5	DATA DICTIONARY	31
	5.1 Overview	32
	5.2 Data Schema	32
	5.2.1 Collected Data Structure	32
	5.2.2 Model Input Tensor	33
	5.3 Data Integrity and Processing	35
	5.4 Dataset Description and Statistics	36
	5.4.1. Dataset Overview	36
	5.4.2. Per-Class Sample Distribution	36
	5.4.3. Dataset Characteristics	

	5.4.4. Data Collection Methodology	38
6	RESULT, DISCUSSION AND CONCLUSION	40
	6.1 Result	41
	6.2 Conclusion	42
7	BIBLIOGRAPHY	44
	7.1 References	45
	7.1.1 Academic Papers & Journals	45
	7.1.2 Technical Documentation	46
	7.1.3 Web Resources & Libraries	46
	7.1.4 Similar Systems (Case Studies)	46

LIST OF FIGURES

NO	NAME	PAGE NO
1	Model Architecture Diagram	24
2	Data Pipeline Flowchart	25
3	Use Case Diagram	26
4	Sequence Diagram (Live Prediction Flow)	27
5	Activity Diagram (Data Collection&Prediction	28

	Workflow)	
6	Context-Level-0 DFD	29
7	Context-Level-1 DFD	30

LIST OF TABLES

NO	NAME	PAGE NO
1	Collected Data Structure	32
2	Model Input Tensor Schema	33
3	Label Encoding Mapping	33
4	Per-Class Classification	36
5	Dataset Characteristics	37

1 INTRODUCTION

1.1 INTRODUCTION

1.2 AIMS AND OBJECTIVE OF THE WORK

1.3 BRIEF LITERATURE REVIEW

1.4 PROBLEM DEFINITION

1.5 PLAN OF THE WORK

1.1. Introduction

Lip reading, also known as visual speech recognition (VSR), is the process of interpreting spoken language by visually analyzing the movements of the lips, face, and tongue. It is a skill naturally employed by hearing-impaired individuals and has gained significant research attention in the fields of computer vision and artificial intelligence. This project presents a Computer Vision Lip Reading System that uses 3D Convolutional Neural Networks (3D CNNs) to automatically recognize spoken words from video sequences of lip movements.

The system operates entirely on visual input—no audio signal is required. A webcam captures the user's face in real-time, and the dlib facial landmark detector identifies 68 key facial points, from which the lip region (landmarks 48-61) is extracted. The system monitors the distance between the upper lip (landmark 51) and lower lip (landmark 57) to detect when the user is speaking. When speech is detected, the system captures a sequence of 22 consecutive lip-region frames, each preprocessed to a standardized 80× 112 pixel RGB image with contrast enhancement and noise reduction.

These 22-frame sequences form the input to a 3D Convolutional Neural Network that has been trained to classify lip movement patterns into one of 13 predefined English words: "here", "is", "a", "demo", "can", "you", "read", "my", "lips", "cat", "dog", "hello", and "bye". The 3D CNN architecture is specifically chosen because it can capture both spatial features (the shape and position of the lips in each frame) and temporal features (how the lip shape changes across the sequence of frames), making it ideal for video-based classification tasks.

The model was trained on a custom-built dataset of approximately 700 video clips (~3 GB of data), collected manually using a purpose-built data collection script. The

training process achieved a 95.7% training accuracy and 98.5% validation accuracy, demonstrating strong classification performance. The system includes both a live prediction mode (real-time webcam inference) and a file-based prediction mode (inference on pre-recorded videos), making it versatile for both interactive and batch processing scenarios.

By leveraging advances in facial landmark detection, image preprocessing, and 3D convolutional neural networks, this project demonstrates that visual-only lip reading is a viable and accurate approach for word-level speech recognition, with applications in assistive technology for the hearing impaired, security and surveillance systems, noisy environment communication, and human-computer interaction research.

1.2. Aims and Objective of the Work

1.2.1. Aims of "Computer Vision Lip Reading System"

The Computer Vision Lip Reading System is designed to recognize spoken words from visual lip movement patterns using deep learning. Based on the project's design and implementation, its primary aims are:

- Visual Speech Recognition: To develop a system that can accurately recognize spoken words solely from visual analysis of lip movements, without requiring any audio input.
- Automated Lip Detection&Frame Capture: To build an intelligent pipeline that automatically detects when a person is speaking by analyzing facial landmark distances, captures the relevant lip-region frames, and preprocesses them for classification.

- Deep Learning Classification: To train a 3D Convolutional Neural Network capable of learning spatiotemporal features from lip movement sequences and classifying them into 13 predefined word categories with high accuracy.
- Real-Time Inference: To enable real-time word prediction from a live webcam feed, providing immediate visual feedback of the recognized word on screen.

In short, the project aims to demonstrate that computer vision and deep learning can be combined to build a practical, visual-only lip reading system that works in real-time.

1.2.2. Objectives of "Computer Vision Lip Reading System"

The specific technical objectives of this project are:

- Implement Facial Landmark Detection: Utilize dlib's pre-trained shape predictor (face_weights.dat) with 68 facial landmarks to detect faces and extract the lip region (landmarks 48-61) from each video frame with sub-pixel accuracy.
- Build a Custom Dataset: Design and implement a data collection pipeline (collect.py) that uses lip-distance thresholds to detect speech onset/offset, captures exactly 22 frames per word (including pre-speech buffer frames), and saves them as structured data for training.
- Develop Image Preprocessing Pipeline: Implement a robust preprocessing chain including LAB color space conversion, CLAHE (Contrast Limited Adaptive Histogram Equalization) for contrast enhancement, Gaussian blurring, bilateral filtering, and sharpening to normalize lip appearance across varying lighting conditions.
- Design and Train a 3D CNN Model: Architect and train a Sequential model with three Conv3D layers (8, 32, 256 filters), MaxPooling3D layers, L2 regularization

(0.001), Dense layers (1024, 256, 64), Dropout (0.5), and softmax output for 13-class classification, achieving>95% validation accuracy.

- Implement Live Prediction System: Build a real-time prediction script (predict_live.py) that captures webcam frames, detects speech activity, accumulates 22 preprocessed lip frames, feeds them through the trained model, and displays the predicted word on screen.
- Implement File-Based Prediction: Build a video file prediction script (predict_file.py) that processes pre-recorded videos frame-by-frame using the same pipeline and saves annotated output videos with predicted word overlays.

1.3. Brief Literature Review

The development of this Lip Reading System is grounded in existing research and innovations across the fields of visual speech recognition, deep learning for video analysis, and facial landmark detection.

1.3.1. Visual Speech Recognition

Visual speech recognition (VSR), also known as automatic lip reading, has been an active area of research since the 1980s. Early systems relied on hand-crafted features such as lip contour shapes and optical flow vectors. Potamianos et al. (2003) provided a comprehensive survey of audio-visual speech recognition, establishing that visual information from lip movements can significantly improve speech recognition accuracy, especially in noisy environments. The GRID corpus (Cooke et al., 2006) became a standard benchmark dataset, containing 34,000 short video clips of 34 speakers, enabling systematic evaluation of VSR systems.

1.3.2. Deep Learning for Lip Reading

The breakthrough application of deep learning to lip reading came with the LipNet model (Assael et al., 2016), which achieved 95.2% accuracy on the GRID corpus using an end-to-end deep learning architecture combining Spatiotemporal CNNs, GRUs, and CTC loss. This was followed by "Lip Reading Sentences in the Wild" (Chung et al., 2017) from DeepMind, which demonstrated that deep learning models could surpass professional human lip readers. The use of 3D Convolutional Neural Networks (3D CNNs) for video analysis was pioneered by Tran et al. (2015) with the C3D model, which showed that 3D convolutions can effectively learn spatiotemporal features from video sequences—the same architectural principle employed in this project.

1.3.3. Facial Landmark Detection

The dlib library's facial landmark detector, based on the work of Kazemi and Sullivan (2014) "One Millisecond Face Alignment with an Ensemble of Regression Trees," provides real-time detection of 68 facial landmarks with high accuracy. This detector is widely used in face recognition, emotion detection, and lip reading applications. The 68-landmark model identifies key points around the jaw, eyebrows, nose, eyes, and mouth—with landmarks 48-67 specifically representing the outer and inner lip contours, which are critical for extracting the lip region of interest (ROI) used in this project.

1.4. Problem Definition

The current landscape of speech recognition and accessibility technology faces several critical challenges that this project seeks to address:

- Audio Dependency of Speech Recognition: Mainstream speech recognition systems (Google Speech-to-Text, Apple Siri, Amazon Alexa) rely entirely on audio input. They fail in noisy environments (factories, concerts, busy streets), underwater, behind glass barriers, or when the speaker is muted. There is no widely available consumer system that can recognize speech from visual input alone.
- Limited Accessibility for Hearing-Impaired: While sign language recognition has received research attention, automated lip reading—which would allow hearing-impaired individuals to understand speakers who do not know sign language—remains an underexplored application area with no mainstream consumer solution.
- Dataset Scarcity for Word-Level Lip Reading: Most existing lip reading datasets (GRID, LRW, LRS) are designed for sentence-level or phoneme-level recognition, often with controlled lighting and studio conditions. There is a scarcity of simple, word-level datasets suitable for training lightweight models that can run in real-time on consumer hardware.
- Real-Time Processing Challenge: Performing facial landmark detection, lip region extraction, image preprocessing, and deep learning inference in real-time from a webcam feed presents computational challenges. The pipeline must be efficient enough to process frames at video rate (15-30 FPS) while maintaining classification accuracy.

1.5. Plan of the Work

To ensure the successful development of the Computer Vision Lip Reading System, the project work is divided into the following strategic phases:

Phase 1: Requirement Analysis&Feasibility

- Reviewing literature on visual speech recognition, 3D CNNs, and facial landmark detection.
- Defining the vocabulary: 13 common English words.
- Determining the technical stack: Python, TensorFlow/Keras, OpenCV, dlib, NumPy.
- Evaluating feasibility of real-time lip reading on consumer hardware.

Phase 2: Data Collection&Dataset Creation

- Designing the data collection pipeline with automatic speech detection via lip-distance thresholds.
- Implementing collect.py with calibration mode, circular frame buffers, and standardized 22-frame output.
- Collecting ~700 video clips across 13 word classes using webcam recordings.
- Organizing data into labeled folders with text files (numpy arrays) and MP4 video files.

Phase 3: Model Design&Training

- Designing the 3D CNN architecture with Conv3D, MaxPooling3D, Dense, and Dropout layers.
- Implementing data loading, label encoding, and train/test split (80/20).
- Training the model for 20 epochs with Adam optimizer and categorical cross-entropy loss.
- Evaluating with confusion matrix, classification report, ROC-AUC curves, and balanced accuracy.

Phase 4: Inference System Development

- Building predict_live.py for real-time webcam-based word prediction.
- Building predict_file.py for batch inference on pre-recorded video files.
- Implementing the same preprocessing pipeline used during training for inference consistency.

Phase 5: Testing&Validation

- Testing with various speakers, lighting conditions, and camera angles.
- Validating model accuracy against the test set (98.5% validation accuracy achieved).

- Recording demonstration videos to showcase system capabilities.

2 TECHNOLOGY AND LITERATURE REVIEW

2.1 TECHNOLOGIES

2.2 LITERATURE REVIEW

2.1. Technologies

2.1.1. Programming Language&Runtime

Python 3 (v3.11/3.12): The primary programming language for the entire project. Python is the de facto standard for machine learning and computer vision projects due to its extensive ecosystem of scientific computing libraries, readable syntax, and strong community support. All data collection, preprocessing, model training, and inference scripts are written in Python.

2.1.2. Deep Learning Framework

TensorFlow (v2.12.0): Google's open-source machine learning framework used as the backend for building, training, and running the 3D CNN model. TensorFlow provides GPU acceleration, automatic differentiation, and production-ready model serving capabilities.

Keras (v2.12.0): The high-level neural network API running on top of TensorFlow. Keras provides the Sequential model API, layer primitives (Conv3D, MaxPooling3D, Dense, Dropout, Flatten), optimizers (Adam), loss functions (categorical_crossentropy), and training utilities (model.fit, model.predict) used throughout the project. Key Keras components used include:

- Conv3D: 3D convolutional layers that apply filters across both spatial (height, width) and temporal (frame sequence) dimensions, enabling the model to learn spatiotemporal features from video sequences.
- MaxPooling3D: Downsampling layers that reduce spatial and temporal dimensions, decreasing computational cost while retaining the most important features.

- Dense: Fully connected layers for classification. The final Dense layer uses softmax activation to output probability distributions across the 13 word classes.
- Dropout: Regularization layers (rate=0.5) that randomly zero out neurons during training to prevent overfitting.
- L2 Regularization (kernel_regularizer): Applied to Conv3D layers with a factor of 0.001 to penalize large weights and improve generalization.

2.1.3. Computer Vision Libraries

OpenCV (v4.6.0.66): The primary computer vision library used for webcam capture (VideoCapture), image color space conversion (BGR to Grayscale, BGR to LAB), image preprocessing (CLAHE, GaussianBlur, bilateralFilter, filter2D for sharpening), image resizing, drawing annotations (circles on landmarks, text overlays), and video writing (VideoWriter for output files).

dlib (v19.24.1): A C++ toolkit with Python bindings used for two critical functions:

- Face Detection: dlib.get_frontal_face_detector() — a HOG+SVM-based frontal face detector that identifies face bounding boxes in grayscale images.
- Facial Landmark Prediction: dlib.shape_predictor() with the pre-trained "shape_predictor_68_face_landmarks" model (face_weights.dat) — predicts 68 facial landmark coordinates, from which lip landmarks (48-61) are extracted for lip region cropping.

Pillow (PIL) (v9.4.0): Python Imaging Library used for creating RGB images from numpy pixel arrays during the data saving process in the collection pipeline.

2.1.4. Data Processing&Utilities

NumPy (v1.23.5): The fundamental numerical computing library used for array operations throughout the project — storing frame sequences as 4D arrays

(frames × height × width × channels), preprocessing kernels (sharpening filter), and model input/output processing.

scikit-learn: Used for data splitting (train_test_split with 80/20 ratio), label encoding (LabelEncoder), evaluation metrics (classification_report, balanced_accuracy_score, confusion_matrix, roc_curve, auc), and label binarization for ROC analysis.

imageio (v2.27.0): Used for reading image files and writing MP4 video files from frame sequences during the data collection process (imageio.mimsave).

matplotlib: Used in the training notebook for plotting training/validation accuracy and loss curves, class distribution histograms, and ROC-AUC curves.

seaborn: Used for generating the confusion matrix heatmap visualization in the training notebook.

2.2. Literature Review

2.2.1. Contextual Background

The Communication Barrier for the Hearing Impaired: The World Health Organization (WHO) reports that over 430 million people worldwide suffer from disabling hearing loss, a number projected to rise to 700 million by 2050. While hearing aids and cochlear implants address some cases, many individuals rely on lip reading as a primary communication method. However, human lip reading is extremely difficult — studies show that even professional lip readers achieve only 40-60% accuracy on unrestricted vocabulary. An automated lip reading system could significantly enhance communication accessibility.

The Rise of Deep Learning in Visual Recognition: The deep learning revolution, beginning with AlexNet (Krizhevsky et al., 2012) winning ImageNet, has transformed computer vision. Convolutional Neural Networks (CNNs) have become the standard for image classification, and their extension to video analysis via 3D CNNs (Tran et al., 2015) has enabled temporal feature learning from video sequences. This project leverages these advances to apply 3D CNNs to the specific problem of lip movement classification.

Audio-Visual Speech Recognition: The McGurk Effect (McGurk&MacDonald, 1976) demonstrated that visual information from lip movements significantly influences speech perception. This finding has driven decades of research into audio-visual speech recognition (AVSR), where visual lip information is fused with audio signals to improve recognition accuracy in noisy conditions (Potamianos et al., 2003).

2.2.2. Existing Solutions&Market Analysis

LipNet (Assael et al., 2016): The first end-to-end deep learning model for sentence-level lip reading, achieving 95.2% accuracy on the GRID corpus. LipNet uses Spatiotemporal CNNs + GRUs + CTC loss, but requires large-scale labeled video datasets and significant computational resources.

DeepMind's Visual Speech Recognition (Chung et al., 2017): "Lip Reading Sentences in the Wild" demonstrated performance surpassing professional human lip readers on the LRS dataset. However, the system requires massive training data (100,000+ utterances) and is designed for sentence-level recognition, making it impractical for lightweight, word-level applications.

Google Speech-to-Text, Apple Siri, Amazon Alexa: These mainstream speech recognition systems operate entirely on audio input and cannot function in visually-observed-only scenarios. They offer no lip reading capability.

Commercial Lip Reading Systems: Companies such as Liopa (UK) have developed lip reading technology for healthcare settings, but these are proprietary, expensive, and designed for specific clinical use cases rather than general-purpose word recognition.

2.3. Problem Statement&Gaps

Accessibility Gap: Despite 430 million people with hearing loss, no mainstream consumer application provides real-time visual lip reading to assist communication. Current assistive technology focuses on hearing aids and sign language translation, leaving lip reading automation as an unmet need.

Complexity Gap: Existing deep learning lip reading systems (LipNet, DeepMind) require massive datasets, sentence-level architectures, and significant computational resources. There is a gap for lightweight, word-level models that can run in real-time on standard consumer hardware with custom-built, smaller datasets.

Dataset Gap: Most lip reading datasets (GRID, LRW, LRS) feature controlled studio conditions and are designed for sentence or phoneme-level recognition. There is a scarcity of word-level datasets collected under natural webcam conditions suitable for training lightweight models.

Audio-Free Gap: The vast majority of speech recognition systems require audio input. A purely visual system that operates without any microphone is needed for scenarios such as: communication through glass barriers, surveillance footage

analysis, space environments, and any context where audio recording is impractical or prohibited.

2.4. The Role of Deep Learning Intervention (The Project's Contribution)

From Audio to Visual: This project shifts speech recognition from audio-dependent to purely visual, demonstrating that a well-designed 3D CNN trained on lip movement sequences can achieve high accuracy (98.5% validation) for word-level classification without any audio signal.

Custom Dataset Creation: The project contributes a novel word-level lip reading dataset of ~700 video clips across 13 words, collected under natural webcam conditions with an automated collection pipeline. This dataset and methodology can be replicated and extended by other researchers.

Lightweight Real-Time Architecture: Unlike heavyweight sentence-level models, this project demonstrates that a relatively simple 3D CNN architecture (3 Conv3D layers + Dense layers) can achieve excellent word-level classification performance while being light enough for real-time webcam inference.

End-to-End Pipeline: The project delivers a complete pipeline from data collection → preprocessing → model training → real-time inference → file-based inference, serving as a practical reference implementation for visual speech recognition research.

Robust Preprocessing: The LAB-CLAHE-Gaussian-Bilateral-Sharpening preprocessing pipeline addresses the real-world challenge of varying lighting conditions during webcam-based lip capture, contributing a practical solution for normalizing lip appearance in uncontrolled environments.

3 SYSTEM REQUIREMENTS STUDY

3.1 USER CHARACTERISTICS

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

3.3 ASSUMPTIONS AND DEPENDENCIES

3.1. User Characteristics

The Computer Vision Lip Reading System is designed for two distinct user groups, each with specific roles and technical aptitudes.

3.1.1. Hearing-Impaired Individuals&Communication Aid Users (Primary Users)

Role: End users who benefit from real-time visual lip reading for communication assistance.

Characteristics:

- Demographics: Individuals with partial or complete hearing loss of all ages.
- Technical Expertise: Low to Medium. The system is designed for simple usage — run a script and speak into the webcam.
- Frequency of Use: Regular (daily conversations, meetings, educational settings).

Key Privileges:

- Use live prediction mode to see real-time word recognition from webcam.
- View predicted words displayed on screen.
- Reset prediction history for new conversations.

3.1.2. Researchers&Developers (Secondary Users)

Role: Users who extend, modify, or build upon the system for research or development.

Characteristics:

- Demographics: Computer science students, machine learning researchers, accessibility technology developers.
- Technical Expertise: High. Familiar with Python, deep learning, and computer vision.
- Frequency of Use: Periodic (during research, model retraining, dataset expansion).

Key Privileges:

- Use data collection mode to record new training data for additional words.
- Retrain the model with expanded datasets using the Jupyter notebook.
- Modify preprocessing parameters and model architecture.
- Use file-based prediction for batch processing of video files.

3.2. Hardware and Software Requirements

3.2.1. Hardware Requirements

Client-Side (User Machine)

- Processor: Intel Core i5 or equivalent (quad-core 2.0 GHz minimum). GPU recommended for training (NVIDIA CUDA-compatible).
- RAM: Minimum 8 GB (16 GB recommended for model training).
- Webcam: Built-in or external USB webcam with minimum 480p resolution (720p recommended).
- Storage: Minimum 5 GB free space (3 GB for dataset, 1 GB for model weights, 1 GB for dependencies).
- Display: Standard monitor for viewing the webcam feed and prediction results.

Training Environment (Optional)

- The training was performed on Kaggle using cloud GPU (NVIDIA Tesla P100 or T4).
- Local training requires an NVIDIA GPU with at least 4 GB VRAM and CUDA toolkit installed.

3.2.2. Software Requirements

Development/Runtime Environment

- Operating System: Windows 10/11, macOS (Catalina or later), or Linux (Ubuntu 18.04+).
- Python: Version 3.11 or 3.12.
- Code Editor: Visual Studio Code, Cursor IDE, or Jupyter Notebook (for training).

Core Dependencies

Package	Version	Purpose
TensorFlow	2.12.0	Deep learning framework for model building and inference
Keras	2.12.0	High-level neural network API (included in TensorFlow)
OpenCV	4.6.0.66	Computer vision: webcam, image processing, video I/O
dlib	19.24.1	Face detection and facial landmark prediction (68 points)
NumPy	1.23.5	Numerical array operations for frame data

Pillow	9.4.0	Image creation from numpy arrays during data saving
imageio	2.27.0	Video file creation from frame sequences
scikit-learn	—	Data splitting, label encoding, evaluation metrics
matplotlib	—	Training curve visualization
seaborn	—	Confusion matrix heatmap visualization
h5py	3.8.0	HDF5 file I/O for model weight storage (.h5 files)

3.3. Assumptions and Dependencies

3.3.1. Assumptions

- Webcam Availability: It is assumed that the user's machine has a functioning webcam accessible via OpenCV's VideoCapture(0). External USB webcams should work but may require driver installation.
- Frontal Face Orientation: The system assumes the user is facing the camera approximately frontally. Profile or extreme angle views will cause the facial landmark detector to fail, resulting in no lip detection.
- Adequate Lighting: It is assumed that the user's face is reasonably well-lit. While the CLAHE preprocessing helps normalize contrast, extremely dark or heavily backlit conditions may degrade accuracy.

- Single Speaker: The current implementation processes one face at a time (the first detected face). It is assumed that only one speaker is present in the camera frame.
- English Words Only: The system is trained exclusively on 13 English words. It assumes all spoken input belongs to this predefined vocabulary; out-of-vocabulary words will be misclassified into the closest known class.
- Consistent Speaking Style: The system assumes a natural speaking pace and mouth movement. Exaggerated or whispered speech may not match the training data distribution.

3.3.2. Dependencies

- dlib Face Weights: The system depends on the pre-trained facial landmark model file "face_weights.dat" (shape_predictor_68_face_landmarks). This file must be present in the /model/ directory. Without it, face detection and lip extraction fail entirely.
- TensorFlow Model Weights: The trained 3D CNN weights file "model_weights.h5" (or "model_weights2.h5") must be present in the /model/ directory. Without pre-trained weights, the system cannot perform prediction.
- OpenCV VideoCapture: The webcam capture depends on OpenCV's VideoCapture class interfacing with the system's camera driver. Compatibility issues may arise on certain Linux distributions or with specific webcam hardware.
- Python Package Compatibility: The project depends on specific versions of TensorFlow, dlib, and OpenCV that must be mutually compatible. TensorFlow 2.12.0 requires specific NumPy and protobuf versions; dlib requires CMake and C++ compiler during installation.

- GPU Drivers (Training Only): If retraining the model locally, the system depends on NVIDIA CUDA drivers and cuDNN library being correctly installed for GPU-accelerated training.

4 SYSTEM DIAGRAMS

4.1 ENTITY-RELATIONSHIP DIAGRAM

4.2 CLASS DIAGRAM

4.3 USE CASE DIAGRAM

4.4 SEQUENCE DIAGRAM

4.5 ACTIVITY DIAGRAM

4.6 DATA FLOW DIAGRAM

4.1. Entity-Relationship Diagram

An Entity-Relationship Diagram (ERD) is a visual representation used to illustrate the structure and relationships between various entities within a system. It employs entities (representing real-world objects or concepts), attributes (properties of these entities), and relationships (connections between entities) to depict how data is organized.

The Computer Vision Lip Reading System ERD consists of the following entities:

- Word Class (identified by class_id): Represents one of the 13 predefined words in the vocabulary. Attributes: class_id, word_label (e.g., "hello", "bye", "cat").
- Video Sample (identified by sample_id): Represents a single collected video clip of a word being spoken. Attributes: sample_id, folder_path, frame_count (22), creation_timestamp. Related to Word Class via word_label (many-to-one: many samples belong to one word class).
- Frame (identified by frame_index within a sample): Represents a single preprocessed lip-region image. Attributes: frame_index (0-21), pixel_data (80× 112× 3 numpy array), image_path (.png). Related to Video Sample (one-to-many: one sample contains 22 frames).
- Model: Represents the trained 3D CNN. Attributes: model_id, architecture_description, weights_file_path, training_accuracy, validation_accuracy. Related to Word Class (classifies into word classes).

[INSERT Fig4.1: Entity-Relationship Diagram HERE]

Fig4.1: Entity-Relationship Diagram

4.2. Class Diagram

A class diagram depicts the structure and relationships of classes/modules within the software system.

The key classes/modules in the system are:

- DataCollector (collect.py): Main data collection module. Properties: detector, predictor, cap (VideoCapture), all_words, curr_word_frames, past_word_frames (deque), LIP_DISTANCE_THRESHOLD, words, labels. Methods: calibrate_lip_distance(), detect_speech(), capture_frames(), saveAllWords().
- Constants (constants.py): Configuration constants. Properties: TOTAL_FRAMES=22, VALID_WORD_THRESHOLD=1, NOT_TALKING_THRESHOLD=10, PAST_BUFFER_SIZE=4, LIP_WIDTH=112, LIP_HEIGHT=80.
- LivePredictor (predict_live.py): Real-time prediction module. Properties: model (Sequential), detector, predictor, cap, label_dict, spoken_already. Methods: preprocess_lip_frame(), detect_speech(), predict_word(), display_result().
- FilePredictor (predict_file.py): File-based prediction module. Properties: model, detector, predictor, label_dict. Methods: main(video_path), process_frame(), save_output_video().
- CNN3DModel (3DCNN.ipynb): Training module. Properties: model (Sequential), input_shape, label_dict. Methods: load_data(), train(), evaluate(), save_weights().

[INSERT Fig4.2: Class Diagram HERE]

Fig4.2: Class Diagram

4.3. Use Case Diagram

A use case diagram depicts the interactions between actors and the system, showcasing functionality from the user's perspective.

Actors:

- Data Collector (Primary Actor): The person who records training data by speaking words into the webcam.
- End User (Primary Actor): The person who uses the live prediction system.
- Webcam Hardware (System Actor): Provides real-time video frames.
- dlib Landmark Detector (System Actor): Provides facial landmark coordinates.
- 3D CNN Model (System Actor): Performs word classification.

Use Cases:

- Calibrate Lip Distance Threshold
- Collect Training Data for a Word
- Start Live Prediction Session
- Detect Face in Frame
- Extract Lip Region
- Detect Speaking Activity
- Capture&Preprocess Lip Frames
- Predict Spoken Word (automated)
- Display Predicted Word
- Run File-Based Prediction
- Save Output Video with Annotations
- Reset Prediction History (press 'q')

[INSERT Fig4.3: Use Case Diagram HERE]

Fig4.3: Use Case Diagram

4.4. Sequence Diagram

A sequence diagram shows the sequence of messages passed between objects over time during an interaction.

Live Prediction Flow Sequence:

1. User → predict_live.py: Launches script

2. predict_live.py → dlib: Loads face_weights.dat

3. predict_live.py → TensorFlow: Loads model_weights.h5

4. predict_live.py → OpenCV: Opens VideoCapture(0)

5. [Loop] For each webcam frame:

    5.1. OpenCV → predict_live.py: Returns frame

    5.2. predict_live.py → dlib: detector(gray) → face bounding boxes

    5.3. predict_live.py → dlib: predictor(gray, face) → 68 landmarks

    5.4. predict_live.py: Calculates lip_distance (landmark 51 to 57)

    5.5. [Decision] lip_distance > 45?

        Yes → Append preprocessed lip_frame to curr_word_frames

        No → Increment not_talking_counter

    5.6. [Decision] not_talking_counter >= 10 AND len(frames) + 4 == 22?

        Yes → Prepend past_word_frames, form 22-frame input

    5.7. predict_live.py → TensorFlow: model.predict(22-frame array)

5.8. TensorFlow → predict_live.py: Probability array for 13 classes

5.9. predict_live.py: argmax → predicted_word_label

5.10. predict_live.py → OpenCV: putText(predicted_word_label) on frame

6. User presses ESC → Script exits

[INSERT Fig4.4: Sequence Diagram HERE]

Fig4.4: Sequence Diagram

4.5. Activity Diagram

An activity diagram models the flow from one activity to another within the system.

Data Collection&Prediction Activity Flow:

1. [Start] → User launches script (collect.py or predict_live.py)

2. → System loads dlib detector, predictor, and model weights

3. → System opens webcam via OpenCV VideoCapture(0)

4. → [Decision] Is this data collection mode?

   Yes → Prompt user for word label and optional lip distance threshold

   No (prediction mode) → Continue directly

5. → [Decision] Need calibration? (No custom distance provided)

   Yes → Measure lip distance for 50 frames with closed mouth → Calculate average threshold

   No → Use custom threshold

6. → [Loop] For each frame from webcam:

→ Convert to grayscale → Detect faces → Extract landmarks

→ Calculate lip distance (landmark 51 to 57)

→ Extract lip region (landmarks 48-54, 50-58) → Resize to 112× 80

→ Preprocess: LAB → CLAHE → GaussianBlur → bilateralFilter → Sharpen → GaussianBlur

→ [Decision] lip_distance > threshold?

Yes (Talking) → Append lip frame to current word buffer

No (Not talking) → Increment not_talking_counter

→ [Decision] Word complete? (not_talking_counter >= 10 AND total frames == 22)

Data Collection → Save word data to /collected_data/ folder

Prediction → Feed 22 frames to model → Display predicted word

7. → User presses ESC → Release webcam → [End]

[INSERT Fig4.5: Activity Diagram HERE]

Fig4.5: Activity Diagram

4.6. Data Flow Diagram

4.6.1. Context-Level-0 DFD

The Context-Level-0 DFD shows the entire system as a single process:

- User provides: Word label (collection mode), Start/Stop commands
- Lip Reading System returns to User: Predicted word, Visual annotations on webcam feed
- Webcam Hardware provides: Raw video frames (BGR)
- File System exchanges: Training data files (data.txt, .png images, .mp4 videos), Model weights (.h5 files)

[INSERT Fig4.6.1: Context-Level-0 DFD HERE]

4.6.2. Context-Level-1 DFD

The Level-1 DFD breaks the system into major processes:

- Process 1: Face&Lip Detection – Receives raw frames, detects face via dlib, extracts 68 landmarks, crops lip region.
- Process 2: Lip Preprocessing – Applies LAB conversion, CLAHE, Gaussian blur, bilateral filter, sharpening to normalize lip frames.
- Process 3: Speech Activity Detection – Monitors lip distance to determine talking/not-talking state, manages frame buffers.
- Process 4: Frame Accumulation – Accumulates 22 preprocessed lip frames (including past buffer frames) for a single word.
- Process 5: Classification (Prediction Mode) – Feeds 22-frame tensor through 3D CNN model, outputs predicted word label.
- Process 6: Data Saving (Collection Mode) – Saves 22-frame sequences as text files, images, and videos to the dataset folder.

- Data Store: /collected_data/ – Stores labeled word folders with frame data.
  /model/ – Stores trained model weights.

[INSERT Fig4.6.2: Context-Level-1 DFD HERE]

4.6.3. Context-Level-2 DFD For Data Collection

Expands Process 1-4 and 6 for the collection pipeline:

- 1.1: OpenCV VideoCapture(0) → Raw BGR frame
- 1.2: cvtColor(BGR2GRAY) → Grayscale frame
- 1.3: dlib detector(gray) → Face bounding boxes
- 1.4: dlib predictor(gray, face) → 68 landmarks
- 2.1: Extract lip region using landmarks 48-54 (horizontal), 50-58 (vertical)
- 2.2: cv2.resize(lip_frame, (112, 80)) → Standardized lip frame
- 2.3: LAB → CLAHE(L channel) → Merge → BGR → GaussianBlur → bilateralFilter → Sharpen → GaussianBlur
- 3.1: Calculate lip_distance = hypot(landmark57 - landmark51)
- 3.2: Compare with LIP_DISTANCE_THRESHOLD
- 4.1: Manage circular past_word_frames buffer (maxlen=4)
- 4.2: Accumulate curr_word_frames during talking
- 4.3: Prepend past_word_frames when word complete → 22 total frames
- 6.1: Save as JSON text file (data.txt)
- 6.2: Save individual .png images
- 6.3: Combine images into .mp4 video

[INSERT Fig4.6.3: Context-Level-2 DFD For Data Collection HERE]

4.6.4. Context-Level-2 DFD For Prediction

Expands Process 5 for the prediction pipeline:

- 5.1: Reshape 22 frames into input tensor: shape (1, 22, 80, 112, 3)
- 5.2: model.predict(input_tensor) → Probability array of shape (1, 13)
- 5.3: np.argmax(prediction) → predicted_class_index
- 5.4: Check spoken_already list → Skip if already predicted
- 5.5: label_dict[predicted_class_index] → predicted_word_label (e.g., "hello")
- 5.6: cv2.putText(frame, predicted_word_label) → Display on screen for 20 frames

[INSERT Fig4.6.4: Context-Level-2 DFD For Prediction HERE]

5 DATA DICTIONARY

5.1 OVERVIEW

5.2 DATA SCHEMA

5.3 DATA INTEGRITY AND PROCESSING

5.1. Overview

The Computer Vision Lip Reading System stores and processes data in two forms: (1) the training dataset stored as files on disk in the /collected_data/ directory, and (2) the in-memory data structures used during real-time collection and prediction. Unlike database-driven applications, this system uses the file system as its primary storage mechanism for training data, and NumPy arrays as the in-memory data format for model input/output. The model weights are stored as HDF5 (.h5) files, and the facial landmark model is stored as a .dat file.

5.2. Data Schema

5.2.1. Collected Data Structure

Each word sample is stored in a dedicated folder within /collected_data/, named as {word_label}_{sample_number}/ (e.g., hello_1/, cat_23/, bye_5/).

File/Item	Data Type	Description
Folder Name	String (e.g., "hello_42")	Named as {label}_{index}. Contains all data for one spoken word sample.
data.txt	JSON (3D array)	JSON-serialized 3D list: [22 frames] [80 rows] [112 cols] [3 channels]. Each value is an integer 0-255 (RGB pixel value).
0.png to 21.png	PNG images (80× 112)	22 individual lip-region frames saved as PNG images. Frame 0 is the earliest frame, frame

		21 is the latest.
video.mp4	MP4 video	All 22 frames combined into a short video clip at the webcam's native FPS for visual review.

Total Dataset Statistics:

Metric	Value
Total Samples	~700 video clips
Total Data Size	~3 GB
Word Classes	13
Frames per Sample	22
Frame Dimensions	80 (height) × 112 (width) × 3 (RGB channels)
Words	"here", "is", "a", "demo", "can", "you", "read", "my", "lips", "cat", "dog", "hello", "bye"

5.2.2. Model Input Tensor

The 3D CNN model expects input in the following tensor format:

Dimension	Size	Description
Batch	1 (inference) / 16 (training)	Number of samples per batch

Frames (Temporal)	22	Number of sequential lip-region frames per word
Height (Spatial)	80	Height of each preprocessed lip frame in pixels
Width (Spatial)	112	Width of each preprocessed lip frame in pixels
Channels	3	RGB color channels

Full input shape: (batch_size, 22, 80, 112, 3)

Label Encoding Mapping:

Encoded Value	Word Label
0	a
1	bye
2	can
3	cat
4	demo
5	dog
6	hello
7	here
8	is
9	lips

10	my
11	read
12	you

5.3. Data Integrity and Processing

Frame Count Consistency: Every word sample is guaranteed to contain exactly 22 frames. The data collection script enforces this through a combination of the PAST_BUFFER_SIZE (4 frames of pre-speech context), active speech frames, and NOT_TALKING_THRESHOLD logic. Samples that do not meet the exact 22-frame requirement are discarded during collection.

Spatial Consistency: All lip frames are resized to exactly 112× 80 pixels using cv2.resize(), regardless of the original lip region size detected by the facial landmarks. This ensures uniform tensor dimensions for the 3D CNN input.

Preprocessing Pipeline: Every frame undergoes the same preprocessing chain:

- LAB Color Space Conversion (cv2.COLOR_BGR2LAB)
- CLAHE Contrast Enhancement (clipLimit=3.0, tileGridSize=(3,3)) on the L channel
- LAB to BGR Conversion back
- Gaussian Blur (7× 7 kernel)
- Bilateral Filter (d=5, sigmaColor=75, sigmaSpace=75)
- Sharpening via custom kernel: [[-1,-1,-1],[-1,9,-1],[-1,-1,-1]]
- Final Gaussian Blur (5× 5 kernel)

Data Types: Raw frames are stored as Python lists of integers (0-255) in JSON format. For model input, they are converted to NumPy float arrays. The model uses float32 internally for all computations. Train/Test Split: The dataset is split 80% training / 20% validation using sklearn's train_test_split with random_state=42 for reproducibility.

6 RESULT, DISCUSSION AND CONCLUSION

6.1 RESULT

6.2 CONCLUSION

6.1. Result

The Computer Vision Lip Reading System has been successfully developed and tested, demonstrating the following functional outcomes:

1. Custom Dataset Creation:

The data collection pipeline (collect.py) successfully captured approximately 700 video clips across 13 word classes, totaling ~3 GB of data. Each clip contains exactly 22 preprocessed lip-region frames at 80× 112 pixels. The automated speech detection based on lip-distance thresholds reliably distinguished speaking from non-speaking states.

2. Model Training Performance:

The 3D CNN model (3 Conv3D layers with L2 regularization, 3 Dense layers with Dropout) was trained for 20 epochs with Adam optimizer and categorical cross-entropy loss, achieving:

- Training Accuracy: 95.7%
- Validation Accuracy: 98.5%
- The validation accuracy exceeding training accuracy suggests effective regularization and a well-generalized model that is not overfitting.

3. Classification Metrics:

Per-class precision, recall, and F1-score were computed using sklearn's classification_report. The confusion matrix heatmap shows strong diagonal dominance, indicating that the model correctly classifies the vast majority of samples for each of the 13 word classes. The ROC-AUC curve demonstrates near-perfect area under the curve for all classes.

[INSERT Training Accuracy/Loss Graph, Confusion Matrix, ROC-AUC Curve, Metrics Table HERE]

4. Real-Time Live Prediction:

The predict_live.py script successfully performs real-time word prediction from a webcam feed. The system detects when the user speaks, captures 22 lip frames, feeds them through the model, and displays the predicted word on screen. The spoken_already mechanism prevents the same word from being predicted consecutively, and pressing 'q' resets the history for a new sentence.

5. File-Based Prediction:

The predict_file.py script successfully processes pre-recorded video files, performs frame-by-frame lip reading, and saves annotated output videos with predicted word overlays. This enables batch processing and demonstration recording.

[INSERT Screenshots – Data Collection Mode, Live Prediction Mode, Sample Lip Frames HERE]

6.2. Conclusion

This project successfully demonstrates that visual-only lip reading using 3D Convolutional Neural Networks is a viable approach for word-level speech recognition. By combining facial landmark detection, robust image preprocessing, and a well-designed 3D CNN architecture, the system achieves 98.5% validation accuracy across 13 word classes—without using any audio information.

The project validates several key technical achievements:

- A custom-built data collection pipeline can efficiently generate high-quality lip reading training data using only a standard webcam and dlib's facial landmark detector.
- 3D CNNs are effective at capturing spatiotemporal features from lip movement sequences, enabling accurate word classification from visual data alone.
- The LAB-CLAHE-Gaussian-Bilateral-Sharpening preprocessing pipeline effectively normalizes lip frame appearance across varying lighting conditions.
- The complete pipeline from data collection → training → real-time inference can run on standard consumer hardware without requiring specialized equipment.

Future enhancements planned for subsequent iterations include:

- Expanding the vocabulary beyond 13 words to support a larger set of common English words.
- Implementing sentence-level lip reading by combining the word-level model with a language model for context-aware prediction.
- Adding speaker-independent training by collecting data from multiple speakers to improve generalization across different face shapes and speaking styles.
- Implementing a graphical user interface (GUI) for easier usage by non-technical users.
- Exploring transfer learning from larger pre-trained video models (e.g., I3D, SlowFast) to improve accuracy with less training data.
- Deploying as a web application or mobile app for broader accessibility.
- Integrating with assistive technology frameworks for hearing-impaired users.

By bridging the gap between audio-dependent speech recognition and visual-only lip reading, this project contributes to the growing body of work demonstrating that

computer vision and deep learning are powerful tools for building accessible communication technology.

7 BIBLIOGRAPHY

7.1 REFERENCES

7.1. References

7.1.1. Academic Papers&Journals

[LipNet] Y. M. Assael, B. Shillingford, S. Whiteson, and N. de Freitas, "LipNet: End-to-End Sentence-level Lipreading," arXiv preprint arXiv:1611.01599, 2016.

[Lip Reading in the Wild] J. S. Chung, A. Senior, O. Vinyals, and A. Zisserman, "Lip Reading Sentences in the Wild," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6447-6456, 2017.

[C3D] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning Spatiotemporal Features with 3D Convolutional Networks," Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 4489-4497, 2015.

[Facial Landmarks] V. Kazemi and J. Sullivan, "One Millisecond Face Alignment with an Ensemble of Regression Trees," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1867-1874, 2014.

[Audio-Visual SR] G. Potamianos, C. Neti, G. Gravier, A. Garg, and A. W. Senior, "Recent Advances in the Automatic Recognition of Audiovisual Speech," Proceedings of the IEEE, vol. 91, no. 9, pp. 1306-1326, 2003.

[McGurk Effect] H. McGurk and J. MacDonald, "Hearing lips and seeing voices," Nature, vol. 264, no. 5588, pp. 746-748, 1976.

[GRID Corpus] M. Cooke, J. Barker, S. Cunningham, and X. Shao, "An audio-visual corpus for speech perception and automatic speech recognition," Journal of the Acoustical Society of America, vol. 120, no. 5, pp. 2421-2424, 2006.

[AlexNet] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Advances in Neural Information Processing Systems (NIPS), pp. 1097-1105, 2012.

[WHO Hearing Loss] World Health Organization, "World Report on Hearing," Geneva: WHO, 2021. (Evidence that 430 million people globally have disabling hearing loss).

7.1.2. Technical Documentation

TensorFlow Documentation. [Online]. Available: https://www.tensorflow.org/api_docs. [Accessed: Jan 2025]. (Deep learning framework for model building, training, and inference).

Keras Documentation. [Online]. Available: https://keras.io/api/. [Accessed: Jan 2025]. (High-level neural network API: Sequential, Conv3D, Dense, Dropout).

OpenCV Documentation. [Online]. Available: https://docs.opencv.org/4.x/. [Accessed: Jan 2025]. (Computer vision library: VideoCapture, image processing, color space conversion).

dlib Documentation. [Online]. Available: http://dlib.net/python/index.html. [Accessed: Jan 2025]. (Face detection and 68-point facial landmark prediction).

NumPy Documentation. [Online]. Available: https://numpy.org/doc/stable/. [Accessed: Jan 2025]. (Numerical computing library for array operations).

scikit-learn Documentation. [Online]. Available: https://scikit-learn.org/stable/. [Accessed: Jan 2025]. (Machine learning utilities: train_test_split, classification metrics).

7.1.3. Web Resources&Libraries

dlib – Face Landmark Detection. [Online]. Available: http://dlib.net/. (C++ toolkit with Python bindings for face detection and shape prediction).

dlib Shape Predictor 68 Landmarks. [Online]. Available: https://github.com/davisking/dlib-models. (Pre-trained facial landmark model used in this project).

OpenCV – Open Source Computer Vision Library. [Online]. Available: https://opencv.org/. (Industry-standard computer vision library used for image/video processing).

Kaggle Lip Reading Dataset. [Online]. Available: https://www.kaggle.com/datasets/allenye66/best-lip-reading-dataset. (The dataset created for this project, hosted on Kaggle).

imageio – Python Library for Image/Video I/O. [Online]. Available: https://imageio.readthedocs.io/. (Used for reading images and writing MP4 videos from frame sequences).

7.1.4. Similar Systems (Case Studies)

LipNet – End-to-End Lip Reading. [Online]. Available: https://github.com/rizkiarm/LipNet. (Open-source implementation of the LipNet sentence-level lip reading model).

Google Speech-to-Text. [Online]. Available: https://cloud.google.com/speech-to-text. (Audio-based speech recognition API — does not support visual lip reading).

Liopa – Visual Speech Recognition. [Online]. Available: https://www.liopa.ai/. (Commercial lip reading technology for healthcare, developed in the UK).

Mozilla DeepSpeech. [Online]. Available: https://github.com/mozilla/DeepSpeech. (Open-source audio speech-to-text engine — audio-only, no visual component).

Computer-Vision-Lip-Reading (v1.0). [Online]. Available: https://github.com/allenye66/Computer-Vision-Lip-Reading. (The predecessor project that this system builds upon, using a different strategy).